Neurocomputing 596 (2024) 127952

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

When decoupled GCN meets group discrimination: A special graph contrastive learning framework

Yinjie Gao [a], Yaxin Sun [a], Shan Jin [b], Dawei Zhang [a], Jinli Cao [c], Zhonglong Zheng [a,*]

[a] School of Computer Science and Technology, Zhejiang Normal University, Jinhua, 321000, China
[b] Polytechnic Institute, Zhejiang University, Hangzhou, 310000, China
[c] Department of Computer Science and Information Technology, La Trobe University, Australia

ARTICLE INFO

Communicated by A.D. Jagtap

Keywords:
Graph Contrastive Learning
Graph Neural Network
Self-supervised learning
Machine learning

ABSTRACT

While Graph Contrastive Learning (GCL) has demonstrated satisfactory outcomes, three main problems remain. First, to effectively transmit the message of higher-order hop nodes, it is imperative to stack multiple graph convolution layers, which results in heightened computational demands and memory requirements. Second, the traditional GCL framework employs the pairwise comparison of positive and negative samples, which could slow model training and necessitate considerable computing resources. Third, the GCL paradigm places considerable emphasis on the data augmentation. Some augmentation techniques focus on altering the graph's structure, which demand significant computational resources. In order to resolve the aforementioned problems in GCL, we incorporate the decoupled GCN into GCL paradigm and develop a new GCL framework termed Decoupled Group Discrimination (DGD), which relies on a one-layer Multi-Layer Perceptron (MLP) encoder and entirely abandons GCN. We also put forward a new negative sample generation strategy which is customized for DGD and enriches the graph structure information during training. Furthermore, we identify a more effective approach to aggregate information from individual nodes by a range of learnable parameters. Experiments on various datasets for the node classification reveal that DGD yields the superior accuracy, faster training time, and less memory consumption. Specifically, DGD improves the overall accuracy of the node classification by a margin of 1 to 2 percentages with the time required for the model training diminished by over 90% and the memory reduced exceeding 70%.

1. Introduction

Graph-structured data are ubiquitous [1,2] and Graph Neural Network (GNN) [3] is a deep learning method that operates upon the graph structure to extract deep network features via multiple layers of neural networks.

This approach facilitates the more comprehensive learning and prediction of the entire graph [4]. While supervised or semi-supervised methods have made significant strides in graph-based deep learning research, the demands and expenses associated with collecting and labeling tags present a significant challenge.

Self-supervised learning [5,6] is a recent innovation which is not reliant on annotations but instead derives labels automatically by uncovering relationships between diverse data elements. Graph Contrastive Learning (GCL) [7] is a graph self-supervised learning method which is premised upon the maximization of mutual information and learns through predicting the consistency between two augmented graph instances [8].

Despite GCL's notable achievements, it is accompanied by three primary issues. First, the employment of the one-layer of GCN in the conventional GCL paradigm restricts the message passing solely to the neighbors within one hop of the nodes. In the pursuit of transmitting high-order hop node information, multiple GCN layers are required, leading to a concomitant rise in computational time and space cost that may eventually decay its performance [9]. Second, the pairwise comparison between positive and negative samples in the contrastive learning leads to a protracted and resource-intensive model training process. Furthermore, GCL heavily relies on data augmentation [10]. It frequently amalgamates two augmentation methods predicating on node attributes and topological structure [11]. However, augmentation methodologies involving graph structure alterations, such as graph diffusion, subgraph sampling, and edge modification [12], necessitate extensive time. The inappropriate sampling of negative samples, and augmentation approaches can further exacerbate the decrease of GCL's performance.

* Corresponding author.
E-mail addresses: gaoyinjie@zjnu.edu.cn (Y. Gao), sunyaxin2005@163.com (Y. Sun), 12321310@zju.edu.cn (S. Jin), davidzhang@zjnu.edu.cn (D. Zhang), J.Cao@latrobe.edu.au (J. Cao), zhonglong@zjnu.edu.cn (Z. Zheng).

https://doi.org/10.1016/j.neucom.2024.127952
Received 20 February 2024; Received in revised form 17 April 2024; Accepted 27 May 2024
Available online 28 May 2024
0925-2312/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Previous research tried decoupling GCN by dissecting the two-step process of the message passing and feature transformation, while also reducing the redundant linear transformations that are inherent in GCN. One example is SGC [9], which is a well-known and seminal work. In the realm of GNNs, the utilization of SGC [9] has primarily been focused on supervised or semi-supervised scenarios. However, the exploration of decoupled GCN in self-supervised scenarios remains unexplored. Consequently, a meaningful research direction arises in the form of designing an appropriate pretext task within the framework of decoupled GCN to effectively extract the supervised signal from the graph. In addition, recently, a novel paradigm of GCL called Graph Group Discrimination (GGD) [13] has been proposed, which discriminates two groups of augmented nodes and corrupted nodes to obtain node representation. While GGD [13] successfully addresses some issues in the conventional GCL framework, it still fails to mitigate the drawbacks of slow computation and resource consumption caused by stacked multi-layer GCN in network structure design.

To address these issues, we adopt a principle according to which GCN can be decoupled as two distinct modules, the message passing process and the multi-layer perceptron (MLP). While the message passing facilitates the transmission of information within the graph, MLP is used to transform the node features. We propose a special GCL model, Decoupled Group Discrimination (DGD), based on this premise. Our model abandons GCN as the primary graph encoder and instead uses an encoder with a single layer of MLP. This step improves the malleability and scalability of our model, enabling us to freely combine numerous hop message passing views for optimal node information transmission within the graph, minimizing the redundancy of feature transformations, and enhancing computational efficiency. Moreover, DGD involves a new negative sample generation strategy termed partial corruption. The partial corruption is adapt to the framework of the decoupled GCN incorporated into Graph Contrastive Learning, which does not necessitate complicated computations but rather constitutes a straightforward yet effective data augmentation strategy exclusive to the proposed DGD framework. This method varies the level of complexity in discriminating positive from negative groups during model training, thereby enabling the model to gain additional useful information from a broader range of structures. Finally, DGD adopts a learnable parameter set upon node representation aggregation to help preserve richer information. When Binary Cross-Entropy (BCE) function applied in the loss computation, the loss computed with the aggregated node embeddings avoids the pairwise comparison of nodes and has faster computational speed. The contributions are as follows:

- We introduce the concept of decoupled GCN to graph contrastive learning methodologies. Specifically, we propose a model named DGD that sheds the use of GCN and instead employs a one-layer MLP encoder, leading to fast training and reduced memory usage.
- We present a fresh negative sampling strategy, referred to as partial corruption, that can be adapted to DGD and enables the model to acquire fine graph structure information during training by regulating the complexity of the learning task.
- We propose an improved approach for node information aggregation that preserves a rich semantic information for individual nodes via a set of learnable parameters.
- We demonstrate the efficacy of DGD in terms of the accuracy, training time, and memory consumption, through extensive experiments on diverse node classification datasets.

2. Related work

2.1. Graph Neural Network

Inspired by the rapid advancements in Convolutional Neural Networks (CNNs), researchers have endeavored to extend the concept of parameter sharing in CNNs to non-European structured data. However, the unique characteristics of graph data present challenges in defining the convolution on graphs. In response to this problem, early researchers proposed two methods: spectral-based and spatial-based approaches. The spectral-based method, similar to CNNs, intuitively realizes convolution by aggregating information from neighboring nodes. This spectral-based approach has propelled the development of Graph Neural Networks (GNNs) and has provided a framework for understanding information transmission in GNNs. The famous work [3] highlights the challenge of migrating the convolutional process from CNN to GCN, which requires determining a fixed neighborhood. GraphSAGE [14] employs a sampling method to determine the neighborhood. This involves performing a fixed number of random walks on the convolution nodes to obtain a fixed-sized set of neighboring nodes, followed by information aggregation.

Building upon prior research, the convolution operation in Graph Convolutional Networks(GCNs) [15] achieves by aggregating the neighboring node information. GCN accesses the first-order neighboring nodes and leverages the first-order hierarchical stacking to obtain information from higher orders, such as second-order and third-order. However, the multi-layer GCN suffers from the over-smoothing problem, which is challenging to mitigate. To address this challenge, GCNII [16] and DeepGCN [17] incorporated the residual connections borrowed from convolutional networks in Computer Vision. GDC [18] utilized the graph diffusion technique to expand the aggregation range of node neighborhoods and obtain global information. At the same time, various decoupled GCN frameworks, such as SGC [9], have been proposed to eliminate non-linear calculations between GCN layers and produce locally smooth features on the graph. In lieu of employing the multi-layer convolution, both APPNP [19] and PPRgo [20] opted to utilize the PPR matrix to effectuate the transmission and aggregation of node information on the graph. Conversely, DAGNN [21] amalgamated diverse hop message passing views with weights that were acquired via the network. Additionally, NAFS [22] eschewed the use of neural networks and instead relied upon attention-like methodologies to compute the weights of different hop views prior to their consolidation. These methods, developed primarily for supervised or semi-supervised learning have mitigated the over-smoothing issue in the original GCN. In this paper, we propose to apply the decoupled GCN to self-supervised learning scenarios and integrate it into a novel contrastive learning framework.

2.2. Graph Contrastive Learning

To overcome the limitations of (Semi-) Supervised Learning's reliance on labels, Self-Supervised learning (SSL) presents a promising approach. The roots of SSL on graphs can be traced back to previous investigations on unsupervised graph embedding, such as DeepWalk [23] and Node2vec [24]. Another noteworthy unsupervised learning model, the Graph Autoencoder [25], can also be regarded as a form of graph self-supervised learning, as it aims to learn the reconstruction of graph structures.

Starting from 2019, the most recent advancements in the graph self-supervised learning have introduced a variety of pre-tasks, ranging from the contrastive learning [26] to graph attribute mining [27]. GCL revolves around the concept of maximizing Mutual Information [28]. These methods encompass early graph embedding techniques [24] based on the random walk [23], as well as the application of a series of computer vision contrastive learning frameworks (such as SimCLR [29] and MoCo [30]) in the field of graphs.

Moreover, in the realm of GCL, various studies have attempted to optimize the technique of data augmentation for adapted and efficient learning [31,32]. GCA [10], CA-GCL [33], and [34] proposed adaptive augmentation schemes, while SimGRACE [35] utilized a perturbed encoder instead of augmentation. These approaches aimed to avoid the complexity of changing topology during augmentation. Other

techniques involved utilizing clustering to resolve negative sample bias, such as method introduced by [36,37]. CuCo [38] optimized the negative sample selection through the curriculum learning, while SUGRL [39] obtained negative samples by directly shuffling embeddings output by MLP. In combination, these studies aimed to develop strategies for optimized negative sample selection. Meanwhile, several other studies combined different pretext tasks [40,41] to provide supervision signals for models from multiple perspectives, such as MLGCL [42], which combined node-level and graph-level contrastive losses, HGMT [43], which utilized both metapath-based encoder and network schema-based encoder to get two different node representations, and MERIT [44], which utilized a Siamese Network to add cross-network contrastive losses. It is different from the above methods that this paper focuses on decoupling the process of GCN's message passing and feature transformation in order to create a more effective and efficient contrastive learning framework.

3. Method

This section will furnish an elaborate explication of the models and techniques posited. First, in Section 3.1, we posit the problem model and briefly highlighted the methodology of data augmentation. Subsequently, in Section 3.2, we apprise the reader on the selection and computation of the message passing view employed. In Section 3.3, we initially expound on GGD [13]'s accomplishments, followed by outlining the specific implementation methodology and the principle of the partial corruption strategy. Further, in Section 3.4, we meticulously elucidate the framework of DGD. Last, in Section 3.5, we provide the inference of the model once its training completed.

3.1. Problem definition

Given a graph $\mathcal{G} = \{\mathbf{X} \in \mathbb{R}^{N\times D}, \mathbf{A} \in \mathbb{R}^{N\times N}\}$, where $\mathbf{X}$ is the node attribute matrix, N is the number of nodes, and D is the dimension of node attributes. $\mathbf{A}$ is the adjacency matrix of the graph. If node i is connected to node j, $\mathbf{A}_{ij} = 1$; otherwise $\mathbf{A}_{ij} = 0$. Motivated by Graph CL [7], we aim to implement the node feature masking as the augmentation technique for a certain graph $\mathcal{G}$. Node feature masking entails setting a predetermined percentage of node feature dimensions and replaces values to zero in randomly selected dimensions. We denote the augmented graph as $\hat{\mathcal{G}} = \{\hat{\mathbf{X}}, \mathbf{A}\}$. The objective is to train an MLP encoder without utilizing node labels. The encoder takes node attributes as input and outputs node feature representations $\mathbf{H} \in \mathbb{R}^{N\times D'}$, where D' is the custom output dimension. $\mathbf{H}$ can be leveraged for various downstream tasks, including the node classification.

3.2. Message passing view

Selection. The initial undertaking involves the selection of the message passing view. Prior literature on decoupled GCN has employed various approaches towards computing weights for aggregating hop-specific message passing views of each node through weighted summation. But DGD, the current model, requires the exchange of node messages for various augmented and corrupted attribute matrices during each epoch of model training. It is essential to consider that the results and computational overhead associated with different hop message passing views and their combinations are different. The most favorable approach, in pursuit of a harmonious blend of model efficacy and computational speed, is to opt for the message passing view with fewer hops for implementation while conducting model training. Therefore, we consider the selected message passing view as a rule that produces correct and incorrect node attributes through correct and incorrect message passing. After weighing the aforementioned factors, we suggest employing a straightforward yet valuable message passing view that does not require extra time to procure.

Computation. The representation of GCN normalized message passing matrix is $\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}$, where $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ is the adjacency matrix of graph $\mathcal{G}$ with self-loop added. $\tilde{\mathbf{D}}$ is the diagonal matrix of $\tilde{\mathbf{A}}$, and $\tilde{\mathbf{D}}_{ii} = \tilde{\mathbf{d}}_i$ is the degree of node i. The present paper employs $\mathbf{A}$ to denote $\hat{\mathbf{A}}$ for the sake of clarity and utility. We posits a message passing view, which may be formulated as follows:

$$\mathbf{X}_n = P(\mathbf{X}_0, \mathbf{A}) = (\mathbf{A} + \lambda\mathbf{I})^n\mathbf{X}_0, \tag{1}$$

where n encapsulates the highest-order hop view that is embedded within the message passing view. $\mathbf{X}_0$ represents the augmented attribute matrix prior to message passing. Typically, higher-order hop views contain amplified information while also entailing greater computational overhead. The parameter λ signifies the incorporation of extra weights for self-connected node information. When the parameter λ is heightened, it is crucial to observe that the message passing process places a heightened emphasis on local neighborhood data. Consequently, in a message passing process characterized by a greater λ value, the central nodes do not expand excessively outward but instead concentrate on information pertinent to the immediate structure. Conversely, nodes situated on the edges encounter more challenges in navigating the intricate network layout. Consequently, an excessively high λ value will impede nodes from acquiring insights into neighbors situated at higher hop distances.

3.3. Partial corruption

Corruption Strategy. The initial step involves the introduction of the fundamental notions of graph homogeneity and graph heterogeneity. These concepts pertain to intrinsic characteristics of graphs and serve to elucidate the extent to which two interconnected nodes exhibit similar attributes, such as belonging to the same category. If the majority of nodes within a graph exhibit comparable characteristics to their neighboring nodes, the graph is considered to possess pronounced homogeneity, thereby being classified as a homogeneous graph. Conversely, a graph featuring a considerable proportion of nodes with dissimilar attributes to those of their neighbors tends to form connections between unrelated nodes, thus earning the classification of a heterogeneous graph. The model in grouping-based contrastive learning employs a corruption strategy to acquire negative samples, predominantly achieved through the random shuffling of the augmented node attribute matrix $\hat{\mathbf{X}}$. The fundamental approach within the corruption strategy involves enhancing the heterogeneity of the graph through modifications to its overarching structure, thereby yielding a heterogeneous graph. The alteration of the node order causes the topological structure of the graph to change correspondingly, resulting in incorrect message passing on the corrupted graph and leading to the creation of negative group. Conversely, correct message passing occurs on the augmented graph, yielding positive group. The model can leverage these two sets of nodes to derive significant structural cues, whereas the conventional GCL typically acquires graph structure details through node neighbor aggregation, focusing more on the distinct attributes of each individual node.

Implement. The present study proposes a methodology, called the partial corruption, for the generation of negative groups. Specifically, given a parameter $\alpha \in (0, 1)$, we divide the augmented node attribute matrix $\hat{\mathbf{X}}$ into two parts according to rows. The first part has $N * \alpha$ rows and the second part has $N * (1 - \alpha)$ rows. It can be described as: $\hat{\mathbf{X}} = (\hat{\mathbf{X}}_1, \hat{\mathbf{X}}_2)$. To this end, partial corruption strategy involves the individual shuffling of $\hat{\mathbf{X}}_1$ and $\hat{\mathbf{X}}_2$, whereby one of them is subject to shuffling while the other remains unaltered. This process results in two corrupted graphs, $\tilde{\mathcal{G}}_1 = \{\tilde{\mathbf{X}}_1, \mathbf{A}\}$ and $\tilde{\mathcal{G}}_2 = \{\tilde{\mathbf{X}}_2, \mathbf{A}\}$, where $\tilde{\mathbf{X}}_1 = (shuffle(\hat{\mathbf{X}}_1), \hat{\mathbf{X}}_2)$, $\tilde{\mathbf{X}}_2 = (\hat{\mathbf{X}}_1, shuffle(\hat{\mathbf{X}}_2))$. Subsequently, message passing is performed on the two resulting graphs $\tilde{\mathcal{G}}_1$ and $\tilde{\mathcal{G}}_2$, applying the message passing view expounded in Section 3.2, thus yielding two distinct attribute matrices, denoted as $\tilde{\mathbf{X}}_{p1} = P(\tilde{\mathbf{X}}_1, \mathbf{A})$, $\tilde{\mathbf{X}}_{p2} = P(\tilde{\mathbf{X}}_2, \mathbf{A})$. The two matrices are subsequently merged, following the separation method applied during

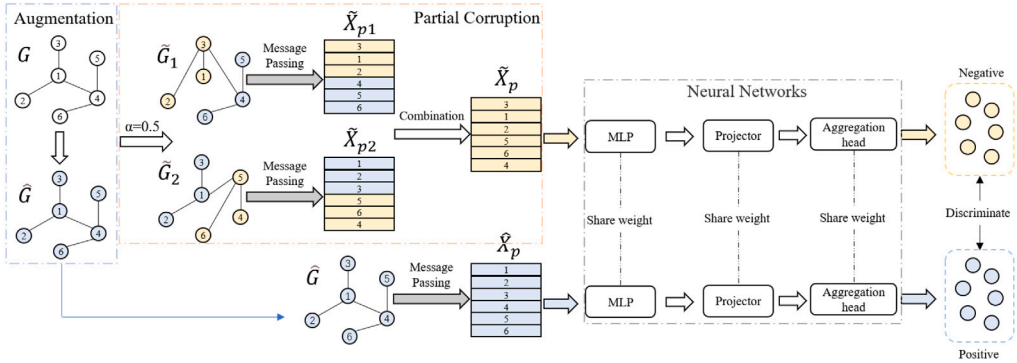

Fig. 1. The architecture of DGD. Given a graph $\mathcal{G}$, we apply augmentation on it to generate an augmented graph $\hat{\mathcal{G}}$. Subsequently, employing the strategy of partial corruption (*e.g.* $\alpha = 0.5$) and executing the message passing and combination procedures, we are able to derive two attribute matrices denoted as $\hat{\mathbf{X}}_p$ and $\tilde{\mathbf{X}}_p$. These two matrices are then regarded as input to a share-weight MLP encoder.

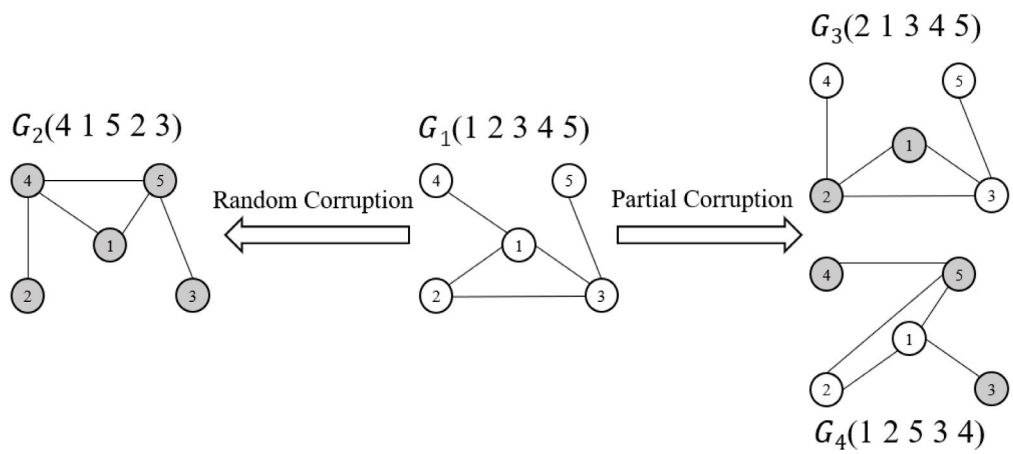

Fig. 2. The comparison between partial corruption and random corruption. The degree of white nodes remains unchanged while the degree of grey nodes changes.

the onset of this methodology, $\tilde{\mathbf{X}}_p = combine(\tilde{\mathbf{X}}_{p1}, \tilde{\mathbf{X}}_{p2})$, where combine operation connects the first $N * \alpha$ rows of $\tilde{\mathbf{X}}_{p1}$ and the last $N * (1-\alpha)$ rows of $\tilde{\mathbf{X}}_{p2}$ into a new matrix. We use $\tilde{\mathbf{X}}_p$ as the attribute matrix of negative group of nodes in DGD. Corresponding to it, the attribute matrix of positive group of nodes is denoted as $\hat{\mathbf{X}}_p = P(\hat{\mathbf{X}}, \mathbf{A})$. The comparison between partial corruption and random corruption is shown in Fig. 2.

Explanation. The graph under consideration contains self-loops, wherein a node can be considered as its own neighbor. Consequently, in the message passing implemented for each node, emphasis is placed on the attributes and degree of their neighboring nodes. Introducing sufficient random node shuffling, the degree and attribute of the neighbors change, but the node retains one neighbor, which is itself, thereby preserving a certain degree of invariance even under conditions of significant perturbations. To increase this level of invariance, we incorporate the additional correct message passing in the incorrect one, thereby augmenting the difficulty for the model in discriminating positive and negative group of nodes. In a partially corrupted node configuration, akin to previous situation, the attributes and degrees of neighboring nodes undergo modifications, whereas the attribute of the node itself remains unaltered. In the event the shuffled node has neighbors that are the same as before the shuffling and belong to the unshuffled portion, the attributes and degrees of those neighbors remain unchanged. Upon examination of the upper right corner of Fig. 2, it is evident that nodes 1 and 2 are shuffled, whereas nodes 3, 4, and 5 retain their original positions. Particularly noteworthy is the status of node 3, which, serving as a common neighbor to nodes 1 and 2, remains unaltered amidst the shuffling. An analysis of the pre and post corruption reveals that the degree and attributes of the node 3, which is the common neighbor of nodes 1 and 2, remain consistent. By maintaining this degree of invariance, the homogeneity of heterogeneous graphs generated through shuffled nodes can be strengthened. This makes it more difficult for the model to group the two sets of nodes, thereby facilitating a more efficient comparison and exploiting the group discrimination's advantage of focusing on the distribution of graph structures.

3.4. Framework

The diagram of DGD is shown in Fig. 1. During the model training, each epoch involves masking of the node feature of a given graph $\mathcal{G} = \{\mathbf{X}, \mathbf{A}\}$ to obtain an augmented graph $\hat{\mathcal{G}} = \{\hat{\mathbf{X}}, \mathbf{A}\}$. According to the given partial corruption parameter α, the partial corruption strategy as prescribed in Section 3.3 is then employed to generate two attribute matrices from partially shuffled graphs $\tilde{\mathcal{G}}_1$ and $\tilde{\mathcal{G}}_2$. Subsequently, three graphs, $\hat{\mathcal{G}}$, $\tilde{\mathcal{G}}_1$ and $\tilde{\mathcal{G}}_2$, are subjected to the message passing view in Section 3.2 to obtain three attribute matrices, represented as $\hat{\mathbf{X}}_p$, $\tilde{\mathbf{X}}_{p1}$ and $\tilde{\mathbf{X}}_{p2}$. $\tilde{\mathbf{X}}_{p1}$ and $\tilde{\mathbf{X}}_{p2}$ are merged in line with the partial corruption strategy to form matrix $\tilde{\mathbf{X}}_p$.

A common neural network is utilized for the input of the augmented node attribute matrix $\hat{\mathbf{X}}_p$ and the corrupted node attribute matrix $\tilde{\mathbf{X}}_p$, comprised of three key components: the primary MLP, a customizable layer projector, and an aggregation head with adjustable learnable parameters. The primary MLP is associated with the non-linear transformation in SGC [9] and represents the node feature transformation component of GCN subsequent to the message passing procedure. The projector can be regarded as an extension of the backbone network within the context of network architecture. Typically,

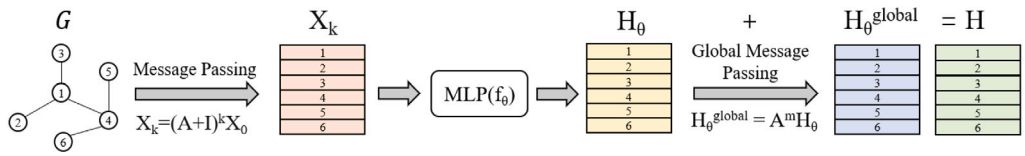

Fig. 3. The inference of DGD. We obtain $\mathbf{X}_k$ following the first message passing, and then put $\mathbf{X}_k$ into trained MLP($\mathbf{f}_\theta$) to derive $\mathbf{H}_\theta$. The second message passing process is executed on $\mathbf{H}_\theta$ to obtain the m-hop global embedding $\mathbf{H}_\theta^{global}$, which is then added to $\mathbf{H}_\theta$ to get final embedding $\mathbf{H}$.

both the backbone network and the projector are structured as single-layer MLP. However, experimental findings suggest that optimizing the number of layers within the projector can potentially amplify the node representations in certain datasets, as demonstrated in the successful implementation of SimCLR [29]. In addition, the aggregation head employs a form of learnable linear aggregation to merge the characteristics of individual nodes based on a defined set of parameters. This process empowers a singular node to preserve more comprehensive semantic information, thus enhancing the overall capability of the model to learn effectively. The method being employed provides a notable level of scalability and adaptability. It allows for the potential adjustment of the output dimension of the aggregation head in anticipation of accommodating multi-classification tasks should the scope of group discrimination expand to encompass discrimination among several groups.

GGD [13] has proved that it is possible to convert general contrastive learning tasks into binary classification tasks. By acquiring the $2N * 1$ predictions for two sets of nodes, we apply the Binary Cross-Entropy (BCE) loss function to classify a total of $2N$ nodes in two groups as positive and negative with corresponding labels of 1 and 0, respectively. The loss for the group discrimination can be articulated as follows:

$$\mathcal{L}_{BCE} = -\frac{1}{2N}\left(\sum_{i=1}^{2N} y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\right), \tag{2}$$

where $y_i = 1$ or $y_i = 0$ is the label of the node i and $\hat{y}_i$ is the prediction of the node i.

3.5. Inference

The inference of DGD is presented in Fig. 3. DGD involves the execution of message passing before feature transformation wherein the original node features are put into the trained encoder following message passing. We perform message passing adhering to the rule as:

$$\mathbf{X}_k = (\mathbf{A}+\mathbf{I})^k\mathbf{X}_0, \tag{3}$$

where $k = 1, 2, 3\ldots$ etc and $\mathbf{X}_0$ is original attribute matrix. The optimal selection of the parameter k is contingent upon the unique characteristics of the graph dataset. Opting for either an excessively small or overly large k value can compromise the model's effectiveness. In situations where the graph consists of a substantial number of nodes, it is advisable to choose a larger k value. Larger graphs necessitate that the nodes communicate with a broader array of remote nodes to acquire adequate global information, thereby justifying the need for a higher k value. The essence concept underlying the message passing view in model inference involves the combination of the $k+1$ hops view $\{\mathbf{X}_0, \mathbf{AX}_0, \mathbf{A}^2\mathbf{X}_0, \ldots, \mathbf{A}^k\mathbf{X}_0\}$, with each view assigning weights based on binomial coefficients. In this manner, we intend to blend diverse hop message passing views for all encompassing consideration of the graph data. Our formula emphasizes low weighting on high-order or low-order hop views, while high weighting is granted to intermediate hop views. This is due to the fact that when the graph is sufficiently large, low-order hop views are restricted by local information with almost no global information in play, while nodes in high-order hop views can be exceedingly smooth. In Section 4.5.3, we investigate the different outcomes produced by utilizing dissimilar weights to amalgamate various hop message passing views. In general, we conduct k-hop message passing on the original graph $\mathcal{G} = \{\mathbf{X}, \mathbf{A}\}$, resulting in $\mathcal{G}_k = \{\mathbf{X}_k, \mathbf{A}\}$. Subsequently, $\mathbf{X}_k$ is inputted to the trained encoder $\mathbf{f}_\theta$ to acquire node representation:

$$\mathbf{H}_\theta = \mathbf{f}_\theta(\mathbf{X}_k). \tag{4}$$

Furthermore, the message passing performed prior to encoder input is synonymous with the message passing procedure of GCN. Following the attainment of the model output embedding, a common method in decoupled GCN is using fixed steps to propagate the acquired features on the graph to aggregate information from multi-hop neighborhoods. This effectively expands the message passing range, leading to improved model output embedding. This methodology is denoted as:

$$\mathbf{H}_\theta^{global} = \mathbf{A}^m\mathbf{H}_\theta, \tag{5}$$

where $\mathbf{H}_\theta^{global}$ is the m-hop global embedding of graph $\mathcal{G}$. The selected value for parameter m is fixed to 10 in our model. The ultimate embedding is attained via

$$\mathbf{H} = \mathbf{H}_\theta^{global} + \mathbf{H}_\theta, \tag{6}$$

which can be utilized for downstream tasks. In Section 4.4, the inclusion of ablation experiments serves to demonstrate the efficacy of this operation in enhancing the quality of the produced model output embedding.

3.6. Algorithm

We have summarized the overall procedure of DGD in Algorithm 1.

Specifically, we first conduct augmentation on $\mathbf{X}$ to obtain $\hat{\mathbf{X}}$. When partial corruption strategy is applied, we can obtain $\tilde{\mathbf{X}}_{p1}$, $\tilde{\mathbf{X}}_{p2}$ and $\hat{\mathbf{X}}_p$ according to the message passing view in model training. Then, we merge $\tilde{\mathbf{X}}_{p1}$ and $\tilde{\mathbf{X}}_{p2}$ to obtain $\tilde{\mathbf{X}}_p$. In the encoding phase, we feed $\hat{\mathbf{X}}_p$ and $\tilde{\mathbf{X}}_p$ to MLP encoder $f_\theta(\cdot)$ and projector $p_\theta(\cdot)$ to generate embeddings $\hat{\mathbf{S}}_\theta$ for positive samples and $\tilde{\mathbf{S}}_\theta$ for negative samples. After that, we concatenate $\hat{\mathbf{S}}_\theta$ and $\tilde{\mathbf{S}}_\theta$ to obtain $\bar{\mathbf{S}}_\theta$, and then we input $\bar{\mathbf{S}}_\theta$ to aggregation head $h_\theta(\cdot)$ to obtain the prediction vector $\hat{\mathbf{y}}_\theta$. The prediction vector $\hat{\mathbf{y}}_\theta$ is a $2N$ dimension vector, which can be used for calculating the loss with a BCE loss. Finally, based on the calculated loss, trainable parameters in $f_\theta(\cdot)$, $p_\theta(\cdot)$ and $h_\theta(\cdot)$ can be updated. This training process will continue iteratively until we reach the predefined number of epochs T. When the training process is completed, we freeze the $f_\theta(\cdot)$ and feed the inference view $\mathbf{X}_k$ to MLP encoder $f_\theta(\cdot)$ to obtain the local embedding $\mathbf{H}_\theta$. Then, we obtain global embedding $\mathbf{H}_\theta^{global} = \mathbf{A}^{10}\mathbf{H}_\theta$. Finally, we obtain final embedding for downstream tasks $\mathbf{H} = \mathbf{H}_\theta^{global} + \mathbf{H}_\theta$.

In DGD framework, the time complexity for the aggregation of a single node is denoted as $O(D')$ prior to conducting loss computation. On the other hand, when aggregating total two groups of nodes, the time complexity extends to $O(2ND')$. The time complexity of the BCE loss for loss computation can be expressed as $O(2N)$ for all nodes within the positive and negative groups, given that the aggregated node embeddings are scalar values. In conclusion, the time complexity of DGD's contrastive learning can be expressed as $O(2ND'+2N)$. The BCE loss effectively moves away from the pairwise comparison of positive and negative groups of nodes, leading to enhanced computational efficiency.

Algorithm 1: The Overall Procedure of DGD

Input : Input node attribute matrix $\mathbf{X} \in \mathbb{R}^{N\times D}$; Normalized adjacency matrix $\mathbf{A} \in \mathbb{R}^{N\times N}$; MLP encoder $f_\theta(\cdot)$; Projector $p_\theta(\cdot)$; Aggregation head $h_\theta(\cdot)$; Number of training epochs T; Number of hidden dimensions D'; Partial corruption parameter α; hop n and parameter λ in the message passing view of model training; hop k in the message passing view of inference;

Output: Final representation $\mathbf{H}$

//Training;
for $t = 1$ *to* T **do**
  Conduct node feature masking on $\mathbf{X}$ to get $\hat{\mathbf{X}}$;
  Get $\tilde{\mathbf{X}}_{p1}$, $\tilde{\mathbf{X}}_{p2}$ and $\hat{\mathbf{X}}_p$ according to Equation (1) in the message passing view and partial corruption strategy in Section 3.3;
  Merge $\tilde{\mathbf{X}}_{p1}$ and $\tilde{\mathbf{X}}_{p2}$ to get $\tilde{\mathbf{X}}_p$;
  Input $\hat{\mathbf{X}}_p$ and $\tilde{\mathbf{X}}_p$ to $f_\theta(\cdot)$ and $p_\theta(\cdot)$ to get embedding $\hat{\mathbf{S}}_\theta = p_\theta(f_\theta(\hat{\mathbf{X}}_p))$ and $\tilde{\mathbf{S}}_\theta = p_\theta(f_\theta(\tilde{\mathbf{X}}_p))$;
  Concatenate $\hat{\mathbf{S}}_\theta$ and $\tilde{\mathbf{S}}_\theta$ to get $\bar{\mathbf{S}}_\theta \in \mathbb{R}^{2N\times D'}$ and then input $\bar{\mathbf{S}}_\theta$ to $h_\theta(\cdot)$ to get $\hat{\mathbf{y}}_\theta$;
  Calculate loss according to Equation (2);
  Update trainable parameters in $f_\theta(\cdot)$, $p_\theta(\cdot)$ and $h_\theta(\cdot)$;
//Inference;
Get $\mathbf{X}_k$ according to Equation (3) in the inference view;
Get local embedding according to Equation (4);
Get global embedding according to Equation (5);
Get final embedding for downstream tasks according to Equation (6);

Table 1
The statistics of seven benchmark datasets.

Dataset	Nodes	Edges	Features	Classes
Cora	2,708	5,429	1,433	7
CiteSeer	3,327	4,732	3,703	6
PubMed	19,717	44,338	500	3
Amazon Computers	13,752	245,861	767	10
Amazon Photos	7,650	119,081	745	8
ogbn-arxiv	169,343	1,166,243	128	40
ogbn-products	2,449,029	61,859,140	100	47

4. Experiments

4.1. Experiment settings

Dataset. The efficacy of our model is measured using a total of seven benchmark datasets, comprising five small-scale datasets, namely Cora, CiteSeer, PubMed [45], Amazon Computers, and Amazon Photos [46], as well as two large-scale datasets, namely ogbn-arxiv and ogbn-products [47]. Relevant statistics pertaining to each of these datasets can be found in Table 1.

Experiment Settings. A single-layered MLP was implemented as the encoder $\mathbf{f}_\theta$ across all datasets. For usage of projector, with an aim to save computational time on various datasets, we minimize the number of projector layer. The present tabular representation outlines the fundamental parameter settings of seven benchmark datasets, in the context of our experimentation. Our study primarily concentrates on fine-tuning six specific parameters for DGD, namely the learning rate, the hidden size, the number of projector layers, the partial corruption parameter α, the self-node information weight λ and the hop k in the view of model inference. The individual parameter configurations adopted for each dataset are explicitly delineated beneath (see Table 2).

Evaluation. The present study involves conducting an evaluation of our self-supervised model by means of node classification tasks. The final node representation $\mathbf{H}$ is utilized to train a logistic regression classifier, which is then applied to each dataset in order to determine its classification accuracy. To ensure reliability of the results, we averaged classification accuracy across 50 experimental trials and used standard deviation for evaluation of model performance. In terms of the training time and memory consumption, we compared DGD with other methods by evaluating the training time and memory consumption required for one epoch. It is acknowledged that DGD necessitates additional time for two message passing calculations during the inference; however, this is deemed acceptable in the comparison to the inevitable iterative calculations required during model training across multiple epochs. Therefore, the time associated with the inference has been listed separately in Section 4.6 and has not been included in the model evaluation.

Table 2
Parameter settings on seven datasets. 'lr' means the learning rate for model training. 'hidden' refers to the hidden size of DGD. 'proj' represents number of layers in projector.

Dataset	lr	hidden	proj	α	λ	k
Cora	1e-3	1024	1	0.2	0	2
CiteSeer	1e-5	1024	3	0.8	1	1
PubMed	1e-3	1024	1	0.1	0	1
Amazon Computers	1e-4	1024	1	0.8	1	4
Amazon Photo	5e-4	1024	1	0.6	1	2
ogbn-arxiv	5e-5	1500	1	–	1	5
ogbn-products	5e-5	1024	1	–	1	22

Table 3
Model performance (*i.e.*, accuracy) of the node classification on five small-scale datasets. Best performance for each dataset is in **bold**. Comp and Photo refer to Amazon Computers and Amazon Photos.

Method	Cora	CiteSeer	PubMed	Comp	Photo
GCN [15]	81.5	70.3	79.0	76.3±0.5	87.3±1.0
GAT [48]	83.0±0.7	72.5±0.7	79.0±0.3	79.3±1.1	86.2±1.5
SGC [9]	81.0±0.0	71.9±0.1	78.9±0.0	74.4±0.1	86.4±0.0
CG3 [49]	83.4±0.7	73.6±0.8	80.2±0.8	79.9±0.6	89.4±0.5
DGI [26]	81.7±0.6	71.5±0.7	77.3±0.6	75.9±0.6	83.1±0.5
GMI [50]	82.7±0.2	73.0±0.3	80.1±0.2	76.8±0.1	85.1±0.1
MVGRL [51]	82.9±0.7	72.6±0.7	79.4±0.3	79.0±0.6	87.3±0.3
GRACE [52]	80.0±0.4	71.7±0.6	79.5±1.1	71.8±0.4	81.8±1.0
GCA [10]	80.5±0.5	71.3±0.4	78.6±0.6	88.9±0.2	91.4±1.0
BGRL [53]	80.5±1.0	71.0±1.2	79.5±0.6	89.2±0.9	91.2±0.8
GBT [54]	81.0±0.5	70.8±0.2	79.0±0.1	88.5±1.0	91.1±0.7
GGD [13]	83.9±0.4	73.0±0.6	81.3±0.8	90.1±0.9	92.5±0.6
DGD	**85.2**±0.3	**74.1**±0.7	**83.5**±0.5	**90.6**±0.3	**94.6**±0.3

Baseline. In this study, we conducted a comparative analysis of DGD against 12 baseline methods on five small-scale datasets. These included four supervised GNNs, namely GCN [15], GAT [48], SGC [9], CG3 [49], and eight advanced GCL methods, namely DGI [26], GMI [50], MVGRL [51], GRACE [52], GCA [10], BGRL [53], GBT [54] and GGD [13]. The data segmentation used for Cora, CiteSeer, and PubMed was the same as that employed in a previous study [55]. For Amazon Computers and Amazon Photos, we utilized semi-supervised experimental settings and randomly assigned 10/10/80% of data to training/validation/testing sets respectively. For ogbn-arxiv, we compared DGD with four self-supervised baselines (DGI [26], GRACE [52], GBT [54], and GGD [13]), while BGRL [53], GBT [54], and GGD [13] were selected for the comparison of ogbn-products. Moreover, our comparative analysis also comprised results from MLP, Node2vec [24], and supervised GCN [15], as originally proposed in a previous work [47]. Certain baseline results reported in this study were obtained from prior research papers [13,44,49].

4.2. Evaluating on small-scale datasets

The present study evaluates the classification accuracy and computational efficiency of a model on five small-scale datasets. To determine the optimal state of the model, we retained the partial corruption and

Table 4
Comparison of training time per epoch in seconds of DGD and other methods on five datasets.

Method	Cora	CiteSeer	PubMed	Comp	Photo
MVGRL [51]	0.123	0.171	0.488	0.663	0.468
GRACE [52]	0.056	0.092	0.893	0.546	0.203
BGRL [53]	0.085	0.094	0.147	0.337	0.273
GBT [54]	0.073	0.072	0.103	0.492	0.173
GGD [13]	0.017	0.019	0.017	0.017	0.016
DGD	**0.013**	**0.016**	**0.015**	**0.017**	**0.014**

Table 5
Node classification result and efficiency comparison on ogbn-arxiv. 'Time' means training time per epoch (in seconds). 'Total' is total training time.

Method	Accuracy (%)	Memory	Time	Total
Supervised GCN [15]	71.7±0.3	–	–	–
MLP	55.5±0.2	–	–	–
Node2vec [24]	70.1±0.1	–	–	–
DGI [26]	70.3±0.2	–	–	–
GRACE [52] (10k epochs)	71.5±0.1	–	–	–
GBT [54] (300 epochs)	70.1±0.2	14,959MB	6.47	1,941
GGD [13] (1 epoch)	71.6±0.5	4513MB	0.18	0.18
DGD (1 epoch)	**71.8**±0.1	**2914MB**	**0.025**	**0.025**

aggregation head components in the comparison of accuracy. To establish comparability with respect to time and memory, GGD [13] and DGD were reproduced based on the same parameter settings provided in [13]. In order to maintain consistency of two models, we kept the identical number of projector layers and sum up each node feature instead of using the aggregation head.

Accuracy. Table 3 presents the classification outcomes for five small-scale datasets. The findings demonstrate that DGD outperforms both supervised GNN and other contemporary GCL baselines across all datasets with respect to accuracy. When compared to the suboptimal method, DGD exhibited a significantly better performance on diverse datasets. Particularly, the advancements observed in Cora, Citeseer, Pubmed, and Amazon Photos datasets ranged from one to two points, while there was a 0.5 point growth in the case of Amazon Computers. The best classification accuracy attests to DGD's superiority and efficacy in generating high-quality node representations.

Time and Memory. GGD [13] has demonstrated that the group discrimination framework experiences a significant increase in training time(3 to 70 times faster) and memory utilization(save 10 to 80 percent) as compared to the traditional GCL method. In order to conduct a fair time and memory comparison experiment, we have reproduced these two models utilizing the same parameter settings as in the aforementioned study. As is evident from Table 4, our results indicate that DGD is almost equivalent to the baseline and corroborates with intuitive understanding. It is worth noting that for small-scale datasets, the one-layer GCN applied in the baseline model is analogous to DGD's one-hop message passing plus a single-layer MLP. Should GGD [13] incorporate more than one layer of GCN for certain datasets, DGD would benefit from significant advantages in terms of time and memory efficiency. Our success is largely attributed to our use of MLP as the backbone network, which eliminates parameter inclusion in message passing and thus reduces redundant linear transformations relative to GCN. It should be noted that our computation time and memory consumption results include additional calculations due to the message passing performed on three node attribute matrices in each epoch in the implementation of DGD.

4.3. Evaluating on large-scale datasets

The present study evaluates the performance of DGD in terms of both classification accuracy and computational efficiency on two large-scale datasets, namely ogbn-arxiv and ogbn-products, which were made

Table 6
Node classification result and efficiency comparison on ogbn-products. Number after \ means the hidden size of GGD [13] and DGD.

Method	Accuracy (%)	Memory	Time	Total
Supervised GCN [15]	75.6±0.2	–	–	–
MLP	61.1±0.0	–	–	–
Node2vec [24]	68.8±0.0	–	–	–
BGRL [53] (100 epochs)	64.0±1.6	29,303MB	53 m 16 s	5,326 m 40 s
GBT [54] (100 epochs)	70.5±0.4	20,419MB	48 m 38 s	4,863 m 20 s
GGD [13] (1 epoch\1024)	75.7±0.4	4,391MB	12 m 46 s	12 m 46 s
DGD (1 epoch\256)	**77.4**±0.2	**1193MB**	**1 m 2 s**	**1 m 2 s**

available by OGB [47]. As for DGD, it employs a full-neighborhood sampling strategy, whereby all one-hop neighbor subgraphs are sampled for a fixed number of nodes per batch, attaining one-hop message passing on each subgraph for said batch of nodes. Because of the comparison on large-scale datasets, to be fair, DGD abstains from utilizing the partial corruption strategy and the aggregation head on both ogbn-arxiv and ogbn-products, instead deploying the same corruption method and summation aggregation as GGD [13].

Tables 5 and 6 present the experimental findings, which demonstrate that DGD has achieved a state-of-the-art classification accuracy with a mere one epoch of training. This indicates that DGD holds superiority in terms of convergence speed due to its ability to achieve effective training within only one epoch. The experimental results show that DGD achieves optimal performance levels during the initial epoch and maintains stable performance with increased epochs. However, BGRL [53] and GBT [54] necessitate a notably higher number of epochs for incremental enhancement of their performance. GGD [13] and DGD exhibit significantly accelerated convergence in comparison to GCL baselines, facilitated by the utilization of the grouping contrastive learning. The reason behind this result could be attributed to the emphasis of grouping-based techniques on the overall edge distribution patterns within graphs, rather than on specific characteristics of individual nodes. Inversely, during the training process, conventional GCL methods may encounter challenges in achieving convergence efficiency due to the potential diversion caused by an excessive focus on node-specific details. Notably, it is worth highlighting that DGD requires only a 256-dimensional embedding on ogbn-products, while surpassing GGD [13]'s 1024-dimensional embedding for two points. DGD's inherent superiority is attributed to the strategy of sampling all one-hop neighbors of batch nodes, along with our message passing and feature transformation separation framework. By generating subgraphs with smaller memory usage, DGD achieves the more complete message passing and better performance.

In regards to the computational efficiency, DGD outperforms other self-supervised baselines in terms of the training time and memory consumption. Specifically, DGD demonstrates a faster training speed than GGD [13]. In the context of the ogbn-arxiv, DGD achieves significant memory savings of approximately 35% compared to GGD [13], with a training speed that is 7 times faster per epoch. Similarly, on the ogbn-products, DGD attains a noteworthy memory reduction of approximately 70%, with a training speed that is 12 times faster per epoch. These observations are attributed to GGD [13] employing multiple layers of GCN, while DGD employs a separated one-hop message passing on these two large-scale datasets.

4.4. Ablation study

An ablation study was conducted on different modules of DGD to determine their effectiveness when implemented separately. It carried out under the same hyperparameter settings across five datasets and the classification accuracy results were averaged across 50 experimental trials. Table 7 presents the outcomes of the study, which demonstrate that all designed modules contribute to the performance of DGD.

Table 7
Ablation Study for DGD. $DGD_{w/o\ p}$ means DGD uses the aggregation head without the partial corruption strategy. $DGD_{w/o\ a}$ means DGD uses the partial corruption strategy without the aggregation head. $DGD_{w/o\ p\&a}$ refers to DGD uses neither the partial corruption strategy nor aggregation head. $DGD_{w/o\ p\&a\&g}$ represents DGD uses no global embedding calculation added in the model inference on the basis of $DGD_{w/o\ p\&a}$.

Method	Cora	CiteSeer	PubMed	Comp	Photo
$DGD_{w/o\ p}$	84.4±0.5	73.7±0.7	83.2±0.5	90.6±0.4	94.5 ±0.3
$DGD_{w/o\ a}$	84.7±0.6	73.9±0.8	82.8±0.4	90.3±0.3	94.4 ±0.3
$DGD_{w/o\ p\&a}$	84.6±0.6	73.6±0.7	82.5±0.4	90.3±0.3	94.4±0.3
$DGD_{w/o\ p\&a\&g}$	83.0±0.8	73.0±0.7	80.4±0.8	90.3±0.3	94.1±0.4
DGD	**85.2**±0.3	**74.1**±0.7	**83.5**±0.5	**90.6**±0.3	**94.6**±0.3

4.4.1. Effectiveness of partial corruption strategy and aggregation head

As it is shown in Table 7, the partial corruption strategy is found to improve or maintain accuracy on all datasets, thus validating its efficacy. Notably, the use of the aggregation head alone results in reduced accuracy on the Cora dataset, but improve performance on other datasets. This observation is potentially due to overfitting issues prevalent in smaller datasets such as Cora. Importantly, the amalgamation of partial corruption strategy with aggregation head results in a noteworthy enhancement and achieves the best accuracy across all datasets. Conversely, disabling both the aggregation head and partial corruption strategy results in the worst accuracy scores across all datasets.

The efficacy of the two proposed modules is thoroughly validated by the experimental outcomes, affirming their potential to enhance the model's performance both individually and in tandem. Furthermore, it is worth noting that the simultaneous operation of these two modules leads to a more significant enhancement in the model's effectiveness. It is our hypothesis that there exists a certain level of interdependence between these two modules, as we have put forth in our proposition. We postulate that partial corruption alters the complexity of model learning, thereby facilitating the acquisition of more relevant information that can be effectively aggregated by parameterized linear layers. So by utilizing an aggregation head, the model is able to acquire and retain the supplementary semantic information of the nodes that is obtained through the partial corruption strategy. Simple summation fails to retain the supplementary acquired knowledge. It is important to highlight that the incorporation of the partial corruption strategy is grounded in the decoupled GCN approach, thereby ensuring a top-down and cohesive nature of our model.

4.4.2. Effectiveness of global embedding calculation

Table 7 demonstrates that removing the global embedding calculation during model inference leads to the poorest accuracy scores across all datasets. Consequently, conducting the computation of global embedding during the inference holds immense significance. This process substantially expands the reach of message passing and enhances the overall quality of the model's output embedding. As a result, downstream tasks can make better use of this improved embedding.

We have conducted comprehensive experiments to ascertain the impact of α on various datasets in Section 4.5.2, which can be employed as a reliable source for selecting α and comprehending the function of the aggregation head.

4.5. Hyperparameter analysis

In this section, we analyze some hyperparameters in the model and do some supplementary experiments to verify the effectiveness of our designs.

4.5.1. Analysis for message passing view in model training

The concept of residual connections has been implemented in the computation of the message passing view in our study. Specifically,

Table 8
Different values of λ in the message passing view of model training on two small-scale datasets. 'Comp' means Amazon Computers.

Dataset	$\lambda = 0$	$\lambda = 0.5$	$\lambda = 1$	$\lambda = 1.5$	$\lambda = 2$	$\lambda = 3$
Cora	84.5±0.7	84.7±0.7	84.5±0.5	84.4±0.9	84.3±0.5	84.1±0.7
Comp	90.1±0.3	90.1±0.4	90.4±0.3	90.3±0.2	90.0±0.4	90.0±0.3

$\mathbf{X}_n = \mathbf{A}^n\mathbf{X}_0$ is identified as the message passing view commonly utilized in decoupled GCN. Furthermore, $\mathbf{A}$ denotes the normalized adjacency matrix while $\mathbf{X}_0$ represents the augmented node attribute matrix. Last, $\mathbf{X}_n$ refers to the node attribute matrix following n-hop message passing. Adhering to the notion of above message passing view, our approach incorporates residual connections, assimilating node attributes from the previous hop into those of the current hop, denoted as

$$\begin{aligned}\mathbf{X}_1 &= \mathbf{A}\mathbf{X}_0 + \mathbf{X}_0\\ &= (\mathbf{A}+\mathbf{I})\mathbf{X}_0,\\ \mathbf{X}_2 &= \mathbf{A}\mathbf{X}_1 + \mathbf{X}_1\\ &= (\mathbf{A}+\mathbf{I})\mathbf{X}_1\\ &= (\mathbf{A}+\mathbf{I})^2\mathbf{X}_0,\\ \mathbf{X}_n &= (\mathbf{A}+\mathbf{I})^n\mathbf{X}_0.\end{aligned} \tag{7}$$

In the context of information exchange among graph nodes, different nodes may possess dissimilar information needs, necessitating the inclusion of a parameter, λ, to regulate the significance of supplementary information transferred from the node to itself. This can be elucidated as

$$\mathbf{X}_n = (\mathbf{A} + \lambda\mathbf{I})^n\mathbf{X}_0. \tag{8}$$

In pursuit of gaining insight into the impact of parameter n and λ, we performed experiments on two small-scale datasets, specifically Cora and Amazon Computers. The results obtained were based on an average accuracy value, derived from 10 separate trials. Notably, the partial corruption strategy and the aggregation head were not utilized in this experimental process, with all other settings being held constant, and only a single hyperparameter was varied.

The experimental findings are presented in Tables 8 and 9. The results suggest that suitable values of λ and n vary across different datasets. The effectiveness of smaller λ values surpasses that of larger values in both datasets. However, the model does not exhibit sensitivity to λ in smaller values. Hence, we speculate that the triumph of the model does not solely rely on the selection of a multi-specific λ. The value of λ that is comparatively diminutive would be deemed fitting in our model. As such, we offer λ values of 0 and 1. The λ value of 0 indicates that the node will not receive any additional information from itself during message passing. Alternatively, the λ value of 1 features a residual connection where the node attributes of the previous hop are appended to the node attributes of the current hop.

Regarding the optimal value of parameter n, it has been observed that on Cora dataset, increasing the value of n leads to a detrimental impact on the model's effectiveness and computational resources. Similarly, on Amazon Computers, augmenting n does not yield any discernible improvement in model efficacy. A plausible explanation for these divergent outcomes is offered by the distinct characteristics of the decoupled GCN applied in our self-supervised model. In traditional decoupled GCN frameworks, the node attributes entering neural network training necessitate various of hop views and their intricate combination. However, DGD achieves commendable outcomes with only one or two hop message passing. Self-supervised model usually derives information by uncovering relationships between diverse data elements. This may mitigate the reliance of our model on the specific message passing view. Therefore, we proposed selecting n fixed as 1 in our investigation.

Based on our aforementioned experiments and subsequent analysis, it can be inferred that utilizing the decoupled GCN approach within a

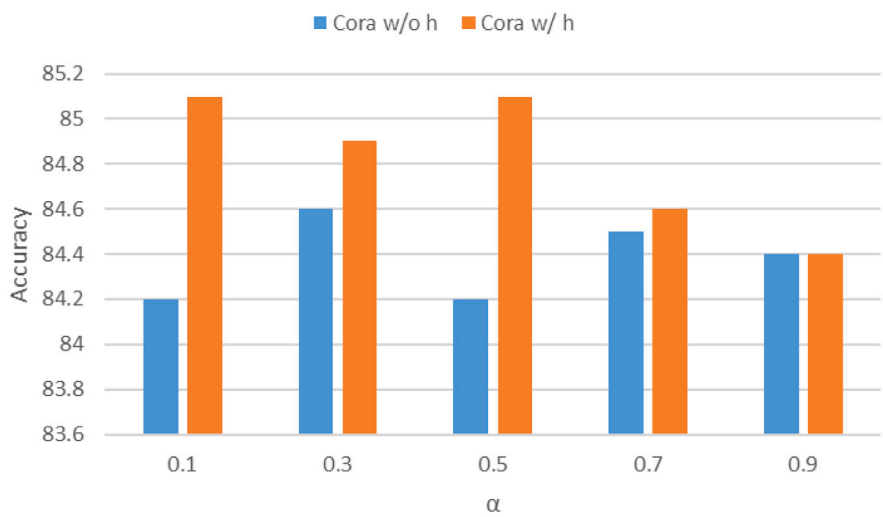

Fig. 4. Different values of α in model training on Cora. $\text{Cora}_{w/o\ h}$ represents a model that does not incorporate an aggregation head. $\text{Cora}_{w/\ h}$ pertains to a model in which an aggregation head has been implemented.

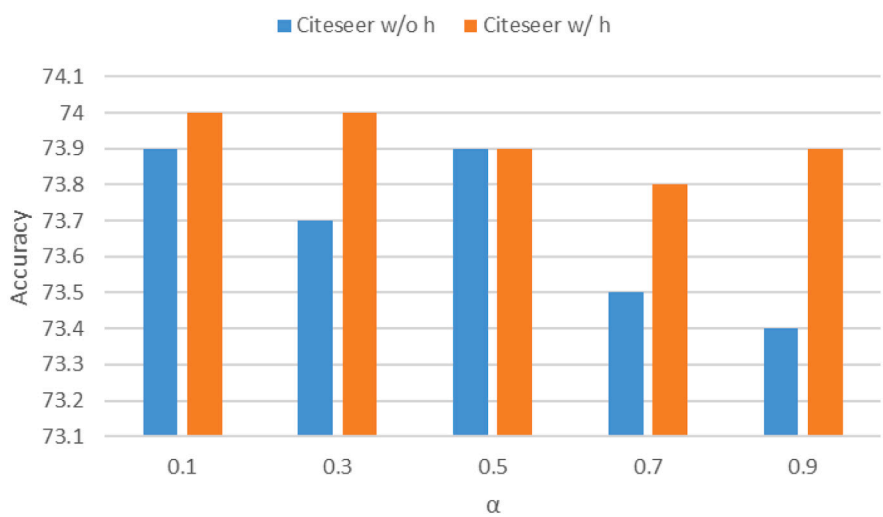

Fig. 5. Different values of α in model training on Citeseer.

Table 9
Different values of n in the message passing view of model training on two small-scale datasets.

Dataset	n = 1	n = 2	n = 3	n = 4
Cora	84.5±0.7	83.8±0.5	83.5±0.4	83.1±0.6
Comp	90.2±0.4	90.4±0.3	90.1±0.4	90.1±0.3

self-supervised model is distinctive in that it is not contingent upon the intricacy of the message passing view. Rather, the fundamental factor is the pretext task that is devised by the model to extract pertinent features from the data. Thus, solely the elected message passing view is implemented as a guiding principle for data processing within our framework. Notably, efficacy is prioritized throughout the system.

4.5.2. Analysis for partial corruption and aggregation head

The present study conducted an investigation into various values of α ranging from 0.1 to 0.9, with a step size of 0.2 on five small-scale datasets. The outcome of the experiment was determined based on an averaged accuracy approach, repeating the process ten times. The resulting outcomes are presented in Figs. 4, 5, 6, 7, and 8.

Based on the obtained results, the ensuing information has been derived. Specifically, the utilization of a learnable aggregation head results in a discernible improvement, or at the very least, sustained performance, as contrasted to the non-utilization of the aggregation head across all α values within every dataset. These findings corroborate our proposal that the aggregation head has the ability to acquire supplementary knowledge from the partial corruption strategy.

Regarding α, it is observed that differing datasets exhibit varying degrees of relatively superior α. α display minimal sensitivity in medium-scale datasets such as Amazon Computers and Amazon Photos, whereas it is comparatively more sensitive in small-scale datasets like Cora, Citeseer and PubMed, which is to be expected. Smaller graphs allow for the selective corruption of key structural through partial corruption strategy, hence leading to significant alterations in results pursuant to varying cutting methods. Notably, on various datasets, certain α values have been known to impair model performance, as their implementation may change the complexity of model training, resulting in models being either too facile or strenuous to learn. Therefore, the employment of α acts as a means of regulating the difficulty of model learning so as to leverage the group discrimination framework to its full potential. Furthermore, our framework's decoupling of GCN from the message passing and feature transformation enables advantages in utilizing the corruption strategy. Finally, if GCN is used as the backbone network, the partial corruption strategy cannot be executed.

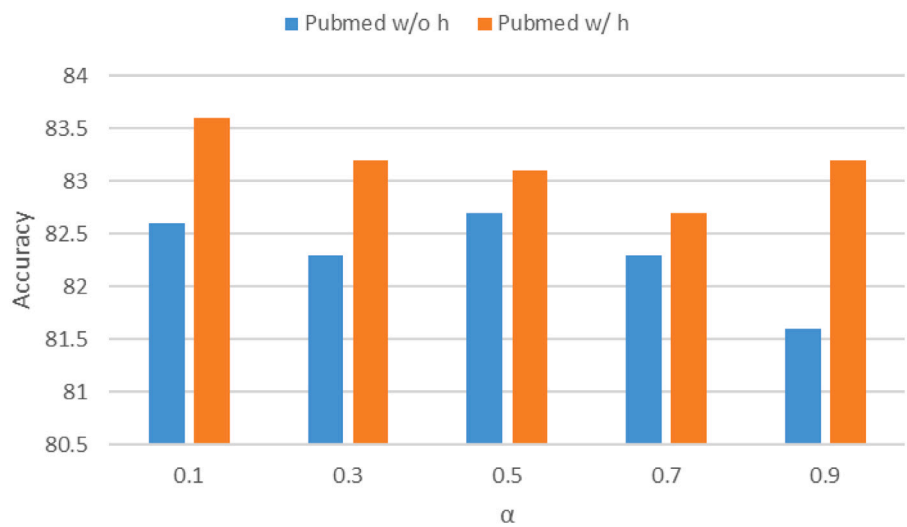

Fig. 6. Different values of α in model training on Pubmed.

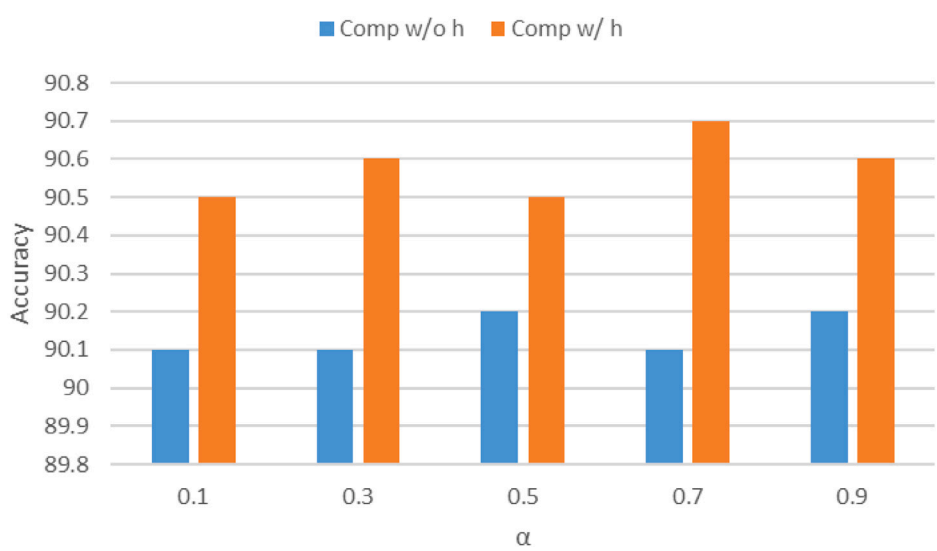

Fig. 7. Different values of α in model training on Amazon Computers.

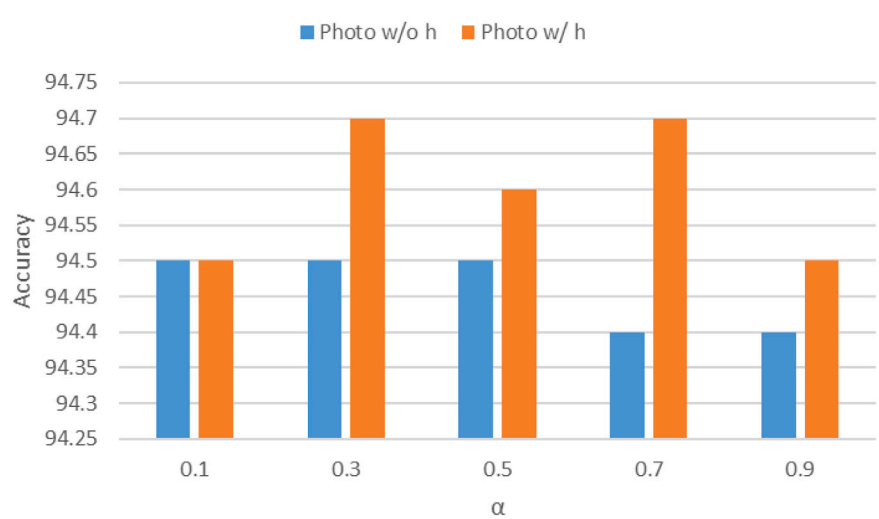

Fig. 8. Different values of α in model training on Amazon Photos.

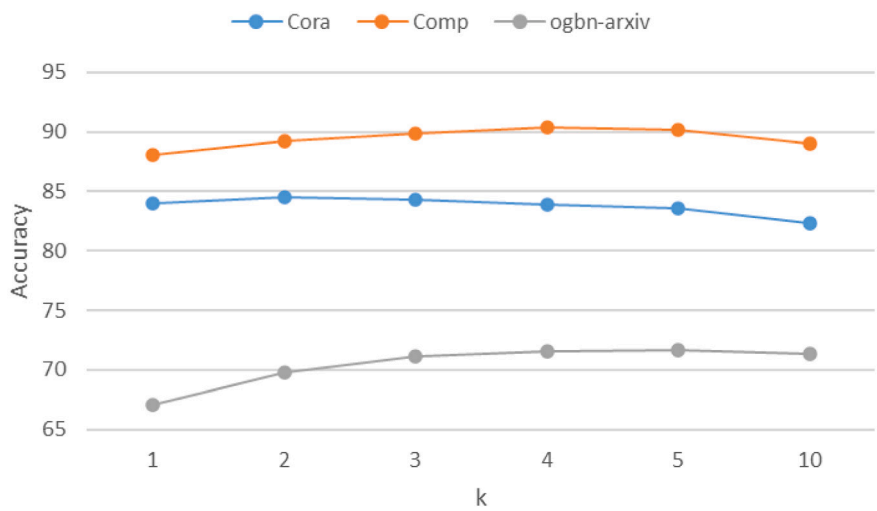

Fig. 9. Different values of k in model inference on three datasets.

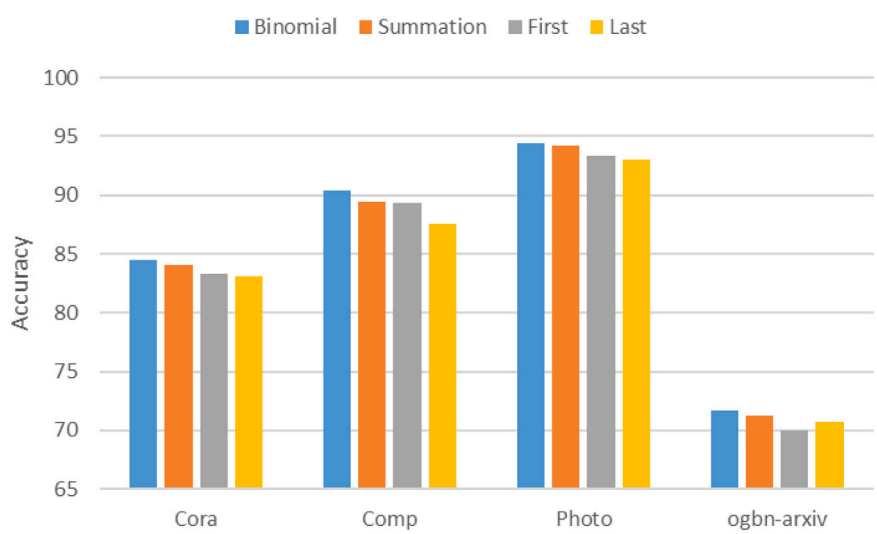

Fig. 10. The utilization of variable weights in the combination of the message passing view in model inference on four datasets. 'First' and 'Last' refer to the one hop view and the highest hop view, respectively.

4.5.3. Supplementary experiments for message passing view in model inference

Through supplementary experiments on the Cora, Amazon Computers, and ogbn-arxiv datasets, we have explored the varying values of the k of the message passing view in model inference. Of the five small-scale datasets, Cora and Amazon Computers are characterized by the least and most number of nodes and edges, respectively, while ogbn-arxiv represents a large-scale dataset. The aforementioned experiments involved the exclusion of both the partial corruption strategy and aggregation head, while holding all other parameters constant. The experimental results were averaged across 10 trials, yielding the reported average accuracy.

It is evident from Fig. 9 that optimal values of k are dependent on the specific dataset being analyzed. The model's efficacy may be compromised by selecting k values that are either too small or too large. Generally, graphs with a greater number of nodes and greater node density necessitate the selection of a larger k. This correlation is intuitive, as a larger graph requires more extensive communication between distant nodes to capture sufficient global information.

In addition, we undertook an investigation into the various weights utilized for combining each hop message passing view during model inference. Initially, we implemented binomial coefficients in our model; then, we subsequently opted to employ different weight combination methodologies for comparative assessment purposes, such as the summation of each hop views, only the first hop view, and solely the last hop view. Supplementary experiments on datasets of Cora, Amazon Computers, Amazon Photos and ogbn-arxiv were also carried out, wherein the partial corruption strategy and aggregation head were not utilized, and all parameters remained unchanged. The experimental outcome was determined by the average accuracy of 10 trials.

The empirical outcomes presented in Fig. 10 offer clear evidence of the exceptional efficacy of the proposed message passing view in the stage of inference, as well as the soundness of our proposed theory regarding the weighted amalgamation of multifarious hop views. Our approach, which employs a straightforward form, garners unparalleled results.

4.6. Time for model inference

On the dataset of ogbn-product, the full graph message-passing process cannot be implemented on NVIDIA GeForce RTX 3070 Laptop GPU. Thus, our investigation solely focuses on assessing the temporal resources invested in model inference involving six distinct datasets namely Cora, Citeseer, Pubmed, Amazon Computers, Amazon Photos

Table 10
Time for model inference in seconds on six benchmark datasets.

Cora	Citeseer	PubMed	Comp	Photo	arxiv
1e-2	1.2e-2	1e-2	1.9e-2	1e-2	3.7e-2

and ogbn-arxiv. In order to conduct the evaluation, we have standardized the hidden size of $\mathbf{H}_\theta$ to 256, and the value of m has been fixed to 10 for all datasets. Detailed outcomes of the experiment are presented below.

The time required for completing the twice message passing and attribute matrix passing through MLP is reflected in Table 10. The inference time is observed to be comparable to that of the training time for a single epoch of the model. This indicates that the efficiency of the approach is acceptable.

It is pertinent to observe that during the model inference, the first time message passing operates on the original node attribute matrix. As a result, the node attribute matrix that is derived following the first time message passing remains unaffected, irrespective of the model parameters utilized during training. This realization implies that the message passing calculation of the initial node attribute matrix can be executed on remote servers, preserved, and subsequently reused. Although our findings include the time of this process, it is imperative to evince its dispensability in practical applications.

5. Conclusion

In this paper, we propose a graph self-supervised learning model called DGD. This work represents the attempt to apply the concept of decoupled GCN to contrastive learning frameworks. DGD eschews GCN entirely, relying solely on a one-layer MLP encoder, thereby greatly improving the training speed and reducing the memory overhead. Furthermore, a new negative sampling strategy termed partial corruption is introduced, which may be adapted to DGD to enhance the learning of complex graph structures by adjusting the level of difficulty of model training. In addition, a more effective method for aggregating single node information is put forward, preserving relatively complete semantic information through a learnable parameter set. Empirical results obtained from diverse node classification datasets demonstrate that DGD outperforms existing state-of-the-art approaches in terms of accuracy, training efficiency, and memory.

CRediT authorship contribution statement

Yinjie Gao: Writing – review & editing, Writing – original draft, Supervision, Methodology, Conceptualization. **Yaxin Sun:** Writing – review & editing, Writing – original draft, Methodology. **Shan Jin:** Writing – review & editing, Supervision, Software, Data curation. **Dawei Zhang:** Writing – review & editing, Supervision. **Jinli Cao:** Writing – review & editing, Supervision. **Zhonglong Zheng:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work was funded by the National Natural Science Foundation of China 62272419, Natural Science Foundation of Zhejiang Province ZJNSFLZ22F020010.

References

[1] Hao Peng, Ruitong Zhang, Shaoning Li, Yuwei Cao, Shirui Pan, S. Yu Philip, Reinforced, incremental and cross-lingual event detection from social messages, IEEE Trans. Pattern Anal. Mach. Intell. 45 (1) (2022) 980–998.
[2] Huanhuan Yuan, Jian Yang, Jiajin Huang, Improving hypergraph convolution network collaborative filtering with feature crossing and contrastive learning, Appl. Intell. (2022) 1–14.
[3] Mathias Niepert, Mohamed Ahmed, Konstantin Kutzkov, Learning convolutional neural networks for graphs, in: International Conference on Machine Learning, PMLR, 2016, pp. 2014–2023.
[4] Xiaowei Zhang, Quan Fang, Jun Hu, Shengsheng Qian, Changsheng Xu, TCKGE: Transformers with contrastive learning for knowledge graph embedding, Int. J. Multimed. Inform. Retriev. 11 (4) (2022) 589–597.
[5] Yaochen Xie, Zhao Xu, Jingtun Zhang, Zhengyang Wang, Shuiwang Ji, Self-supervised learning of graph neural networks: A unified review, IEEE Trans. Pattern Anal. Mach. Intell. 45 (2) (2022) 2412–2429.
[6] Chao Li, Xinming Liu, Yeyu Yan, Zhongying Zhao, Qingtian Zeng, HetGNN-SF: Self-supervised learning on heterogeneous graph neural network via semantic strength and feature similarity, Appl. Intell. (2023) 1–18.
[7] Yuning You, Tianlong Chen, Yongduo Sui, Ting Chen, Zhangyang Wang, Yang Shen, Graph contrastive learning with augmentations, Adv. Neural Inf. Process. Syst. 33 (2020) 5812–5823.
[8] Runliang Tian, Hongmei Shi, Momentum memory contrastive learning for transfer-based few-shot classification, Appl. Intell. 53 (1) (2023) 864–878.
[9] Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, Kilian Weinberger, Simplifying graph convolutional networks, in: International Conference on Machine Learning, PMLR, 2019, pp. 6861–6871.
[10] Yanqiao Zhu, Yichen Xu, Feng Yu, Qiang Liu, Shu Wu, Liang Wang, Graph contrastive learning with adaptive augmentation, in: Proceedings of the Web Conference 2021, 2021, pp. 2069–2080.
[11] Han Zhao, Xu Yang, Cheng Deng, Dacheng Tao, Unsupervised structure-adaptive graph contrastive learning, IEEE Trans. Neural Netw. Learn. Syst. (2023).
[12] Yixin Liu, Ming Jin, Shirui Pan, Chuan Zhou, Yu Zheng, Feng Xia, S. Yu Philip, Graph self-supervised learning: A survey, IEEE Trans. Knowl. Data Eng. 35 (6) (2022) 5879–5900.
[13] Yizhen Zheng, Shirui Pan, Vincent Lee, Yu Zheng, Philip S. Yu, Rethinking and scaling up graph contrastive learning: An extremely efficient approach with group discrimination, Adv. Neural Inf. Process. Syst. 35 (2022) 10809–10820.
[14] Will Hamilton, Zhitao Ying, Jure Leskovec, Inductive representation learning on large graphs, Adv. Neural Inf. Process. Syst. 30 (2017).
[15] Thomas N. Kipf, Max Welling, Semi-supervised classification with graph convolutional networks, 2016, arXiv preprint arXiv:1609.02907.
[16] Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, Yaliang Li, Simple and deep graph convolutional networks, in: International Conference on Machine Learning, PMLR, 2020, pp. 1725–1735.
[17] Guohao Li, Matthias Muller, Ali Thabet, Bernard Ghanem, Deepgcns: Can gcns go as deep as CNNS? in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2019, pp. 9267–9276.
[18] Johannes Gasteiger, Stefan Weißenberger, Stephan Günnemann, Diffusion improves graph learning, Adv. Neural Inf. Process. Syst. 32 (2019).
[19] Johannes Gasteiger, Aleksandar Bojchevski, Stephan Günnemann, Predict then propagate: Graph neural networks meet personalized pagerank, 2018, arXiv preprint arXiv:1810.05997.
[20] Aleksandar Bojchevski, Johannes Gasteiger, Bryan Perozzi, Amol Kapoor, Martin Blais, Benedek Rózemberczki, Michal Lukasik, Stephan Günnemann, Scaling graph neural networks with approximate pagerank, in: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2020, pp. 2464–2473.
[21] Meng Liu, Hongyang Gao, Shuiwang Ji, Towards deeper graph neural networks, in: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2020, pp. 338–348.
[22] Wentao Zhang, Zeang Sheng, Mingyu Yang, Yang Li, Yu Shen, Zhi Yang, Bin Cui, A simple yet tough-to-beat baseline for graph representation learning, in: International Conference on Machine Learning, 2022.
[23] Bryan Perozzi, Rami Al-Rfou, Steven Skiena, Deepwalk: Online learning of social representations, in: Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2014, pp. 701–710.
[24] Aditya Grover, Jure Leskovec, node2vec: Scalable feature learning for networks, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 855–864.
[25] Thomas N. Kipf, Max Welling, Variational graph auto-encoders, 2016, arXiv preprint arXiv:1611.07308.
[26] Petar Veličković, William Fedus, William L. Hamilton, Pietro Liò, Yoshua Bengio, R. Devon Hjelm, Deep graph infomax, 2018, arXiv preprint arXiv:1809.10341.
[27] Yu Rong, Yatao Bian, Tingyang Xu, Weiyang Xie, Ying Wei, Wenbing Huang, Junzhou Huang, Self-supervised graph transformer on large-scale molecular data, Adv. Neural Inf. Process. Syst. 33 (2020) 12559–12571.

[28] R. Devon Hjelm, Alex Fedorov, Samuel Lavoie-Marchildon, Karan Grewal, Phil Bachman, Adam Trischler, Yoshua Bengio, Learning deep representations by mutual information estimation and maximization, 2018, arXiv preprint arXiv:1808.06670.
[29] Ting Chen, Simon Kornblith, Mohammad Norouzi, Geoffrey Hinton, A simple framework for contrastive learning of visual representations, in: International Conference on Machine Learning, PMLR, 2020, pp. 1597–1607.
[30] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, Ross Girshick, Momentum contrast for unsupervised visual representation learning, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 9729–9738.
[31] Lirong Wu, Haitao Lin, Cheng Tan, Zhangyang Gao, Stan Z. Li, Self-supervised learning on graphs: Contrastive, generative, or predictive, IEEE Trans. Knowl. Data Eng. (2021).
[32] Ouyang Jia, Huimin Huang, Jiaxin Ren, Luodi Xie, Yinyin Xiao, Contrastive learning with text augmentation for text classification, Appl. Intell. (2023) 1–10.
[33] Dexuan Lin, Xuefeng Ding, Dasha Hu, Yuming Jiang, Community-aware graph contrastive learning for collaborative filtering, Appl. Intell. (2023) 1–14.
[34] Wen-Zhi Li, Chang-Dong Wang, Jian-Huang Lai, S. Yu Philip, Towards effective and robust graph contrastive learning with graph autoencoding, IEEE Trans. Knowl. Data Eng. (2023).
[35] Jun Xia, Lirong Wu, Jintao Chen, Bozhen Hu, Stan Z. Li, Simgrace: A simple framework for graph contrastive learning without data augmentation, in: Proceedings of the ACM Web Conference 2022, 2022, pp. 1070–1079.
[36] H. Zhao, X. Yang, Z. Wang, E. Yang, C. Deng, Graph debiased contrastive learning with joint representation clustering, in: International Joint Conference on Artificial Intelligence, 2021.
[37] Zhizhong Huang, Jie Chen, Junping Zhang, Hongming Shan, Learning representation for clustering via prototype scattering and positive sampling, IEEE Trans. Pattern Anal. Mach. Intell. (2022).
[38] G. Chu, X. Wang, C. Shi, X. Jiang, CuCo: Graph representation with curriculum contrastive learning, in: International Joint Conference on Artificial Intelligence, 2021.
[39] Yujie Mo, Liang Peng, Jie Xu, Xiaoshuang Shi, Xiaofeng Zhu, Simple unsupervised graph representation learning, in: Proceedings of the AAAI Conference on Artificial Intelligence, AAAI, 2022, pp. 7797–7805.
[40] Xinyi Xu, Cheng Deng, Yaochen Xie, Shuiwang Ji, Group contrastive self-supervised learning on graphs, IEEE Trans. Pattern Anal. Mach. Intell. 45 (3) (2022) 3169–3180.
[41] Qingfeng Li, Huifang Ma, Ruoyi Zhang, Wangyu Jin, Zhixin Li, Dual-view co-contrastive learning for multi-behavior recommendation, Appl. Intell. (2023) 1–18.
[42] Pengpeng Shao, Tong Liu, Dawei Zhang, Jianhua Tao, Feihu Che, Guohua Yang, Multi-level graph contrastive learning, 2021, arXiv preprint arXiv:2107.02639.
[43] Shuai Ma, Jian-wei Liu, Self-supervised contrastive learning for heterogeneous graph based on multi-pretext tasks, Neural Comput. Appl. 35 (14) (2023) 10275–10296.
[44] Ming Jin, Yizhen Zheng, Yuan-Fang Li, Chen Gong, Chuan Zhou, Shirui Pan, Multi-scale contrastive siamese networks for self-supervised graph representation learning, 2021, arXiv preprint arXiv:2105.05682.
[45] P. Sen, G. Namata, M. Bilgic, L. Getoor, Tina Eliassi-Rad, Collective classification in network data, AI Mag. (2008).
[46] Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, Stephan Günnemann, Pitfalls of graph neural network evaluation, 2018, arXiv preprint arXiv:1811.05868.
[47] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, Jure Leskovec, Open graph benchmark: Datasets for machine learning on graphs, 2020, arXiv preprint arXiv:2005.00687.
[48] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, Yoshua Bengio, Graph attention networks, 2017, arXiv preprint arXiv:1710.10903.
[49] Sheng Wan, Shirui Pan, Jian Yang, Chen Gong, Contrastive and generative graph convolutional networks for graph-based semi-supervised learning, in: Proceedings of the AAAI Conference on Artificial Intelligence, 2021, pp. 10049–10057.
[50] Zhen Peng, Wenbing Huang, Minnan Luo, Qinghua Zheng, Yu Rong, Tingyang Xu, Junzhou Huang, Graph representation learning via graphical mutual information maximization, in: Proceedings of the Web Conference 2020, 2020, pp. 259–270.
[51] Kaveh Hassani, Amir Hosein Khasahmadi, Contrastive multi-view representation learning on graphs, in: International Conference on Machine Learning, PMLR, 2020, pp. 4116–4126.
[52] Yanqiao Zhu, Yichen Xu, Feng Yu, Qiang Liu, Shu Wu, Liang Wang, Deep graph contrastive representation learning, 2020, arXiv preprint arXiv:2006.04131.
[53] Shantanu Thakoor, Corentin Tallec, Mohammad Gheshlaghi Azar, Mehdi Azabou, Eva L. Dyer, Remi Munos, Petar Veličković, Michal Valko, Large-scale representation learning on graphs via bootstrapping, 2021, arXiv preprint arXiv:2102.06514.
[54] Jure Zbontar, Li Jing, Ishan Misra, Yann LeCun, Stéphane Deny, Barlow twins: Self-supervised learning via redundancy reduction, in: International Conference on Machine Learning, PMLR, 2021, pp. 12310–12320.
[55] Zhilin Yang, William Cohen, Ruslan Salakhudinov, Revisiting semi-supervised learning with graph embeddings, in: International Conference on Machine Learning, PMLR, 2016, pp. 40–48.

Yinjie Gao obtained a bachelor's degree from Wenzhou University in 2021 and is currently pursuing a master's degree at Zhejiang Normal University. His research interests include machine learning, data mining and pattern recognition.

Yaxin Sun received the B.Eng. degree from the China Three Gorges University, China in 2008, and the Ph.D. degree from South China University of Technology, China in 2015. He is currently a postdoctor of the School of Computer Science, Zhejiang Normal University, China. His research interests include speech emotion recognition, pattern recognition, and machine learning.

Shan Jin obtained a bachelor's degree from Zhejiang Normal University in 2006, a master's degree from Lanzhou University of Technology in 2022, and is currently pursuing a doctoral degree at Zhejiang University. His research interests include machine learning, data mining, analysis techniques, and application development.

Dawei Zhang received the Ph.D. degree from the Department of Computer Science, Zhejiang Normal University, Jinhua, China, in 2022. He is currently working as an assistant professor with the School of Computer Science and Technology, Zhejiang Normal University, Jinhua, China. His research interests include deep learning, computer vision, and multimedia computing.

Jinli Cao, an IEEE member, obtained her Ph.D. in computer science from the University of Southern Queensland, Australia, in 1998. Presently, she holds the position of Associate Professor at the Department of Computer Science and Information Technology, La Trobe University, Melbourne, Australia. Her research interests span the evolutionary fields of data engineering, big data analytics, data privacy protection, system vulnerability prediction, and recommendation systems.

Zhonglong Zheng received the B.Eng. degree from the University of Petroleum, China in 1999, and the Ph.D. degree from Shanghai JiaoTong University, China in 2005. He is currently a full professor and the dean of the School of Computer Science, Zhejiang Normal University, China. His research interests include machine learning, computer vision, and IOT technology with their applications.